

Day 4 시간표

시간	3일차	4일차	
9:30	Ex 4 발표 및 피드백	3.10 Testability Tactics (17min) / 3.11 Usability Tactics (12min) 3.12 Tactics and Architectural Patterns (21min)	
10:00	Ex 5. Architecture View 실습 2 (Documenting Component Level View)		
10:30		Ex 6 발표 및 피드백	
11:00		Ex 7. Architecture Tactic 실습 2 (Performance~ Usability 중심설계)	
11:30			3.1 Architectural Tactics (17min) 3.2 Availability Tactics (38min)
12:00			
12:30	점심 식사	점심 식사	
13:00			
13:30	3.3 Deployability Tactics (27min) 3.4 Energy Efficiency Tactics (13min) 3.5 Integrability Tactics (27min) 3.6 Modifiability Tactics (14min)	Ex 7 발표 및 피드백	
14:00		4-1 Architecture Design (22min) 4-2 Architecture Design (34min)	
14:30			
15:00		Ex 5 발표 및 피드백	Ex 8. Architecture 설계 실습
15:30	Ex 6. Architecture Tactic 실습 1 (Availability~ Modifiability 중심 설계)		
16:00			
16:30			
17:00			
17:30	.7 Performance Tactics (16min)	Ex 8 발표 및 피드백	
18:00	3.8 Safety Tactics(17min) / 3.9 Security Tactics (17min)	과정 정리	

시간	4일차
9:30	3.10 Testability Tactics (17min) / 3.11 Usability Tactics (12min) 3.12 Tactics and Architectural Patterns (21min)
10:00	
10:30	Ex 6 발표 및 피드백
11:00	Ex 7. Architecture Tactic 실습 2 (Performance~Usability 중심설계)
11:30	
12:00	
12:30	
12:30	점심 식사
13:00	
13:30	Ex 7 발표 및 피드백
14:00	4-1 Architecture Design (22min) 4-2 Architecture Design (34min)
14:30	
15:00	Ex 8. Architecture 설계 실습
15:30	
16:00	
16:30	
17:00	Ex 8 발표 및 피드백
17:30	
18:00	과정 정리

Day 4 · 오전

온라인 영상 학습

09:30 ~ 10:30

- 3.10 Testability Tactics (17분)
- 3.11 Usability Tactics (12분)
- 3.12 Tactics and Architectural Patterns (21분)

※ 영상은 1.2 배속으로 학습하세요.

시간	4일차
9:30	3.10 Testability Tactics (17min) / 3.11 Usability Tactics (12min) 3.12 Tactics and Architectural Patterns (21min)
10:00	
10:30	Ex 6 발표 및 피드백
11:00	Ex 7. Architecture Tactic 실습 2 (Performance~Usability 중심설계)
11:30	
12:00	
12:30	점심 식사
13:00	
13:30	Ex 7 발표 및 피드백
14:00	4-1 Architecture Design (22min) 4-2 Architecture Design (34min)
14:30	
15:00	Ex 8. Architecture 설계 실습
15:30	
16:00	
16:30	
17:00	Ex 8 발표 및 피드백
17:30	
18:00	과정 정리

Day 4 · 오전

Ex7 실습 시간

10:30 ~ 12:30

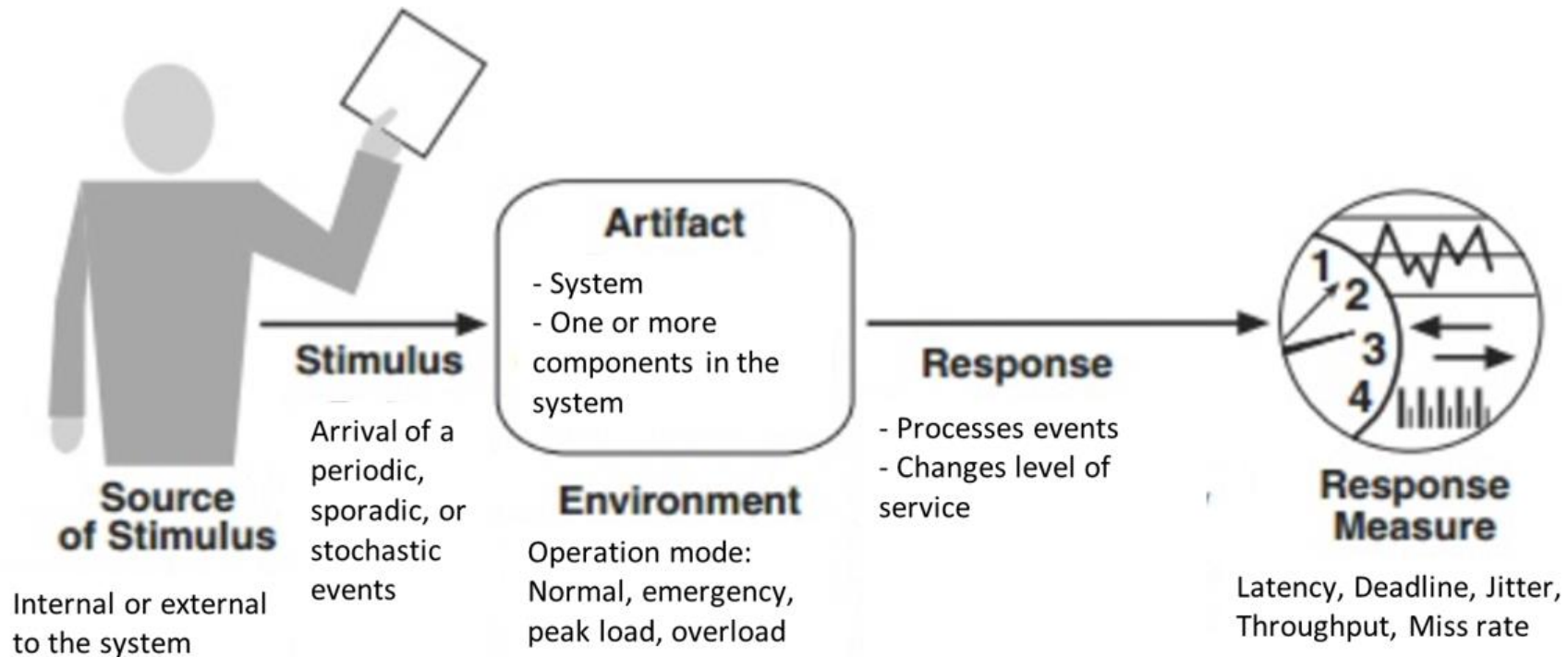
- Architecture Tactic 실습 2 — Performance · Safety · Security · Testability · Usability 품질속성 시나리오 도출 및 Tactic 후보 평가 (조별)

Performance

■ Performance

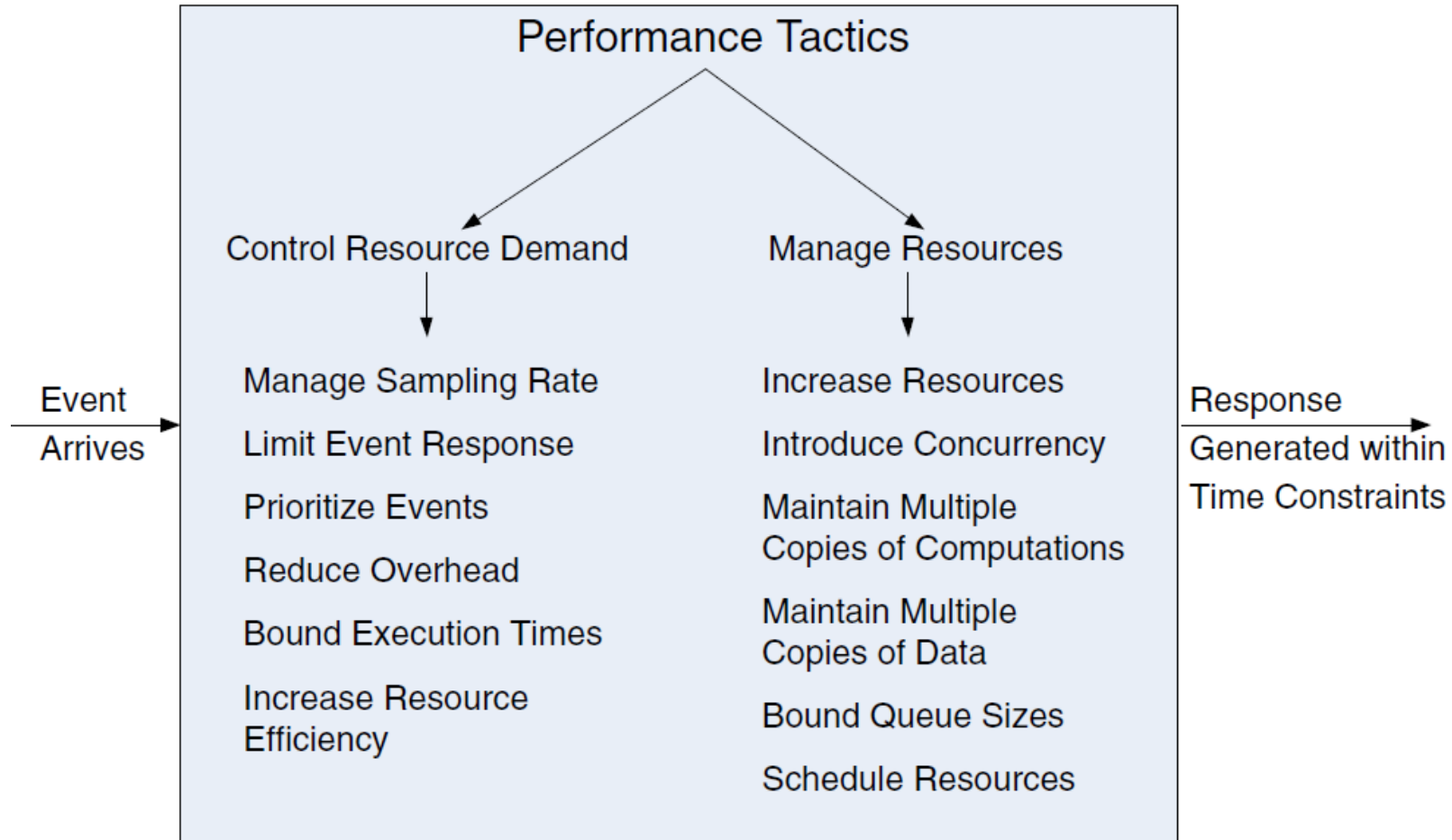
- When events occur, the system **must respond to them in time** 이벤트 발생 시 시스템이 제때 응답하는 능력
- **Arrival pattern for events** : Periodic, Stochastic, Sporadic
- **Time-based response** to those events : Latency, Jitter, Throughput, Miss rate

■ Performance General Scenario



Performance Tactics

- 이벤트 도착(Event Arrives) 후 시간 제약 안에 응답을 생성하기 위한 Tactics — **Control Resource Demand** / **Manage Resources**



Safety

■ Safety

- 시스템의 잘못된 동작이나 장애로 인해 인명, 재산, 환경 등에 피해가 발생하지 않도록 하는 능력

■ Safety General Scenario (1/2)

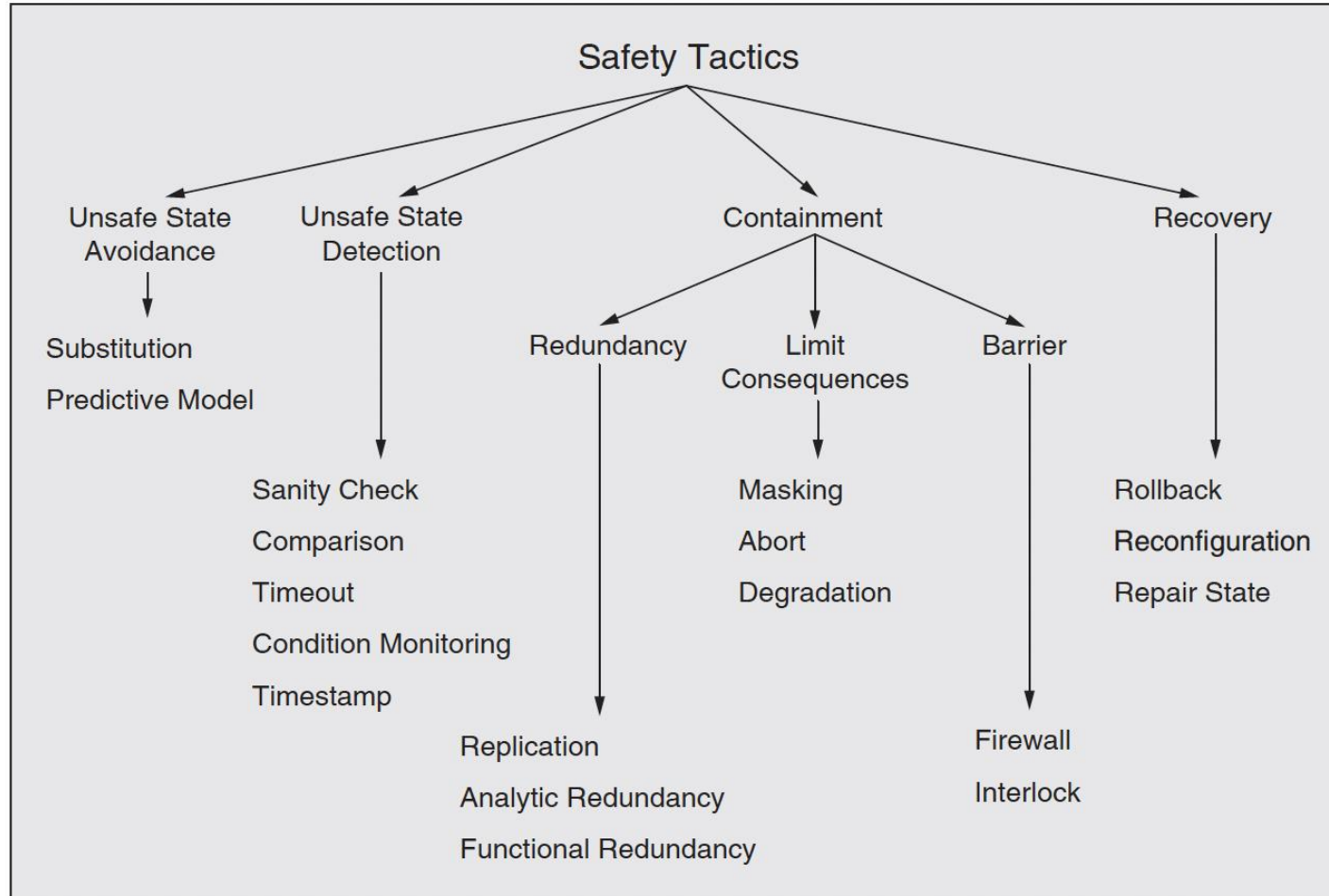
부분	설명	가능한 값
자극원	시스템의 잘못된 동작이나 장애를 일으키는 요인의 주체	다음 중 하나 이상이 가능하다. ▪ 센서 ▪ 소프트웨어 컴포넌트 ▪ 통신 채널 ▪ 장치 (예: 시계)
자극	시스템의 잘못된 동작이나 장애를 일으키는 요인 (예: 생략, 실수, 타이밍 오류)	누락의 구체적인 사례 — 값이 도착하지 않음 / 기능이 수행되지 않음 오발의 구체적인 사례 — 기능이 잘못 수행됨 / 장치가 허위 이벤트를 생성함 타이밍 오류의 구체적인 사례 — 데이터가 너무 늦게나 너무 일찍 도착 / 이벤트가 잘못된 순서로 일어남 값의 오류의 구체적인 사례 — 소프트웨어 컴포넌트가 잘못된 결과를 생성

Safety General Scenario (2/2)

부분	설명	가능한 값
대상	시스템의 어떤 부분이 자극에 영향을 받는가?	▪ 전체 시스템 ▪ 특정 컴포넌트 집합
환경	어떤 상태에서 자극이 발생하는가?	▪ 정상 운영 ▪ 성능 저하 운영 ▪ 수동 운영 ▪ 복구 모드
응답	자극에 대한 반응	다음 중 하나 이상이 가능하다. ▪ 안전하지 않은 상태를 회피한다 ▪ 복구한다 ▪ 저하 혹은 안전 모드로 진행한다 ▪ 섯다운한다 ▪ 수동 운전/백업 시스템으로 전환한다 ▪ 적절한 대상(사람 혹은 시스템)에 보고한다 ▪ 안전하지 않은 상태를 기록한다
응답 측정	응답을 어떻게 측정하는가?	다음 중 하나 이상이 가능하다. ▪ 안전하지 않은 상태로의 진입을 회피한 횟수나 비율 ▪ 안전하지 않은 상태 중에 시스템이 자동으로 회복 가능한 상태의 비율 ▪ 시스템이 복구 가능한 시간 ▪ 시스템이 저하 모드나 안전 모드로 동작하는 시간 ▪ 수동 운영/안전 모드/저하 모드로 진입 후 복구될 때까지 걸린 시간

Safety Tactics

- Unsafe State Avoidance / Unsafe State Detection / Containment / Recovery 의 4 범주

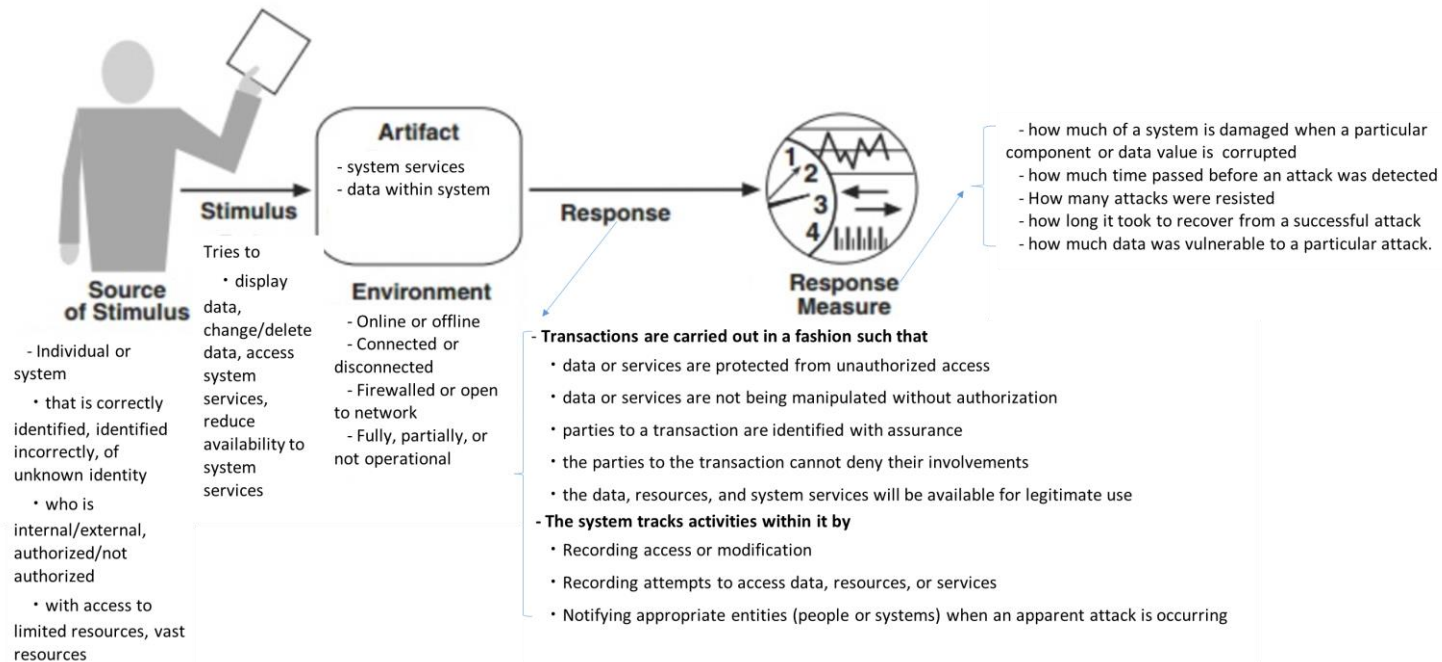


Security

■ Security

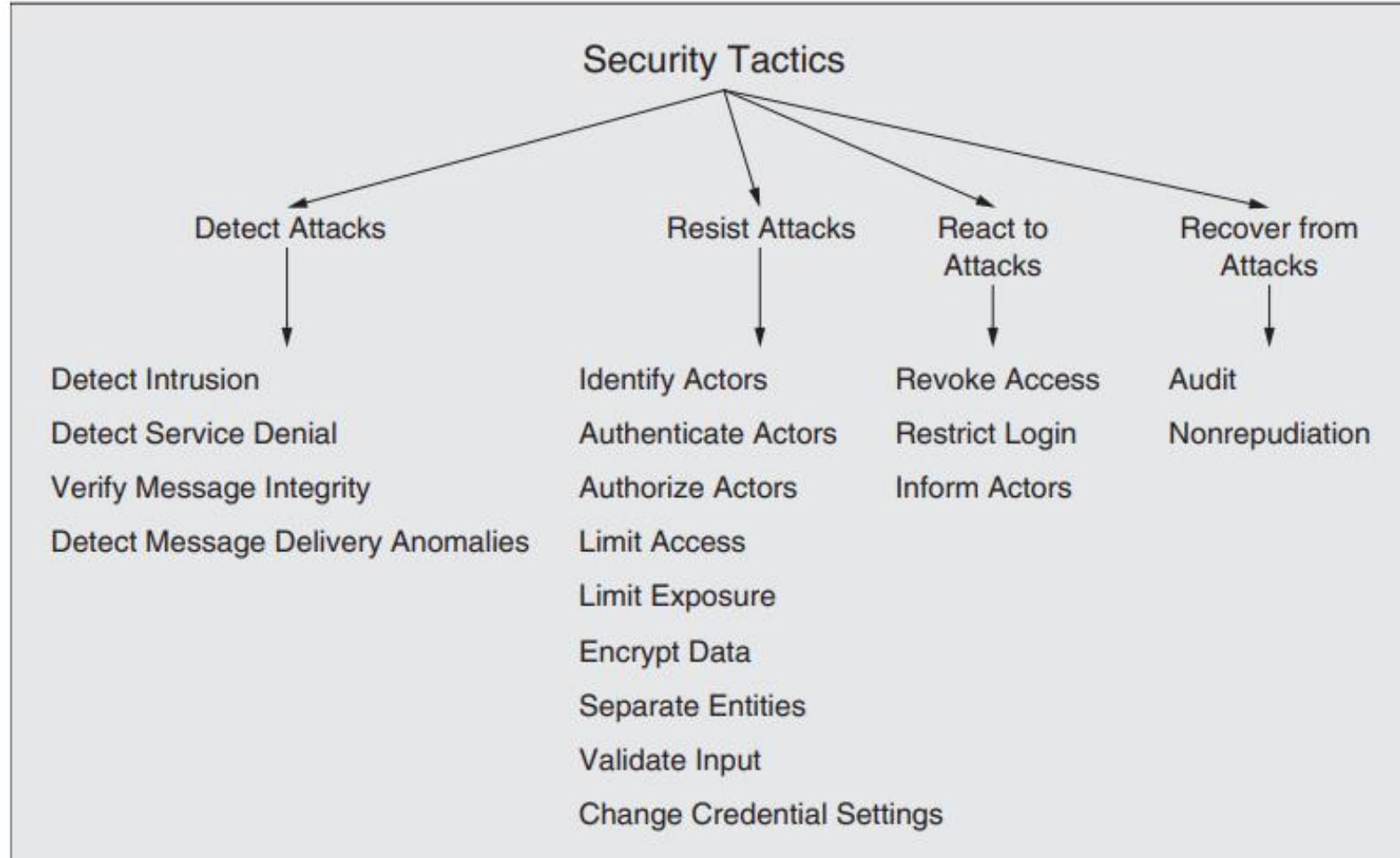
- 승인 받지 않은 접근으로부터 데이터와 정보를 보호하는 능력
 - 기밀성 (**Confidentiality**) : 인가 받지 않은 접근에서 데이터와 서비스를 보호
 - 무결성 (**Integrity**) : 인가 받지 않은 접근에서 데이터와 서비스가 조작되지 않음
 - 가용성 (**Availability**) : 합법적인 사용에 가용함

■ Security General Scenario



Security Tactics

- Detect Attacks / Resist Attacks / React to Attacks / Recover from Attacks 의 4 범주



Testability

■ Testability (시험 용이성)

- 소프트웨어가 테스트를 통해서 오류를 발견하고 수정하기 쉬운 정도

■ Testability General Scenario (1/2)

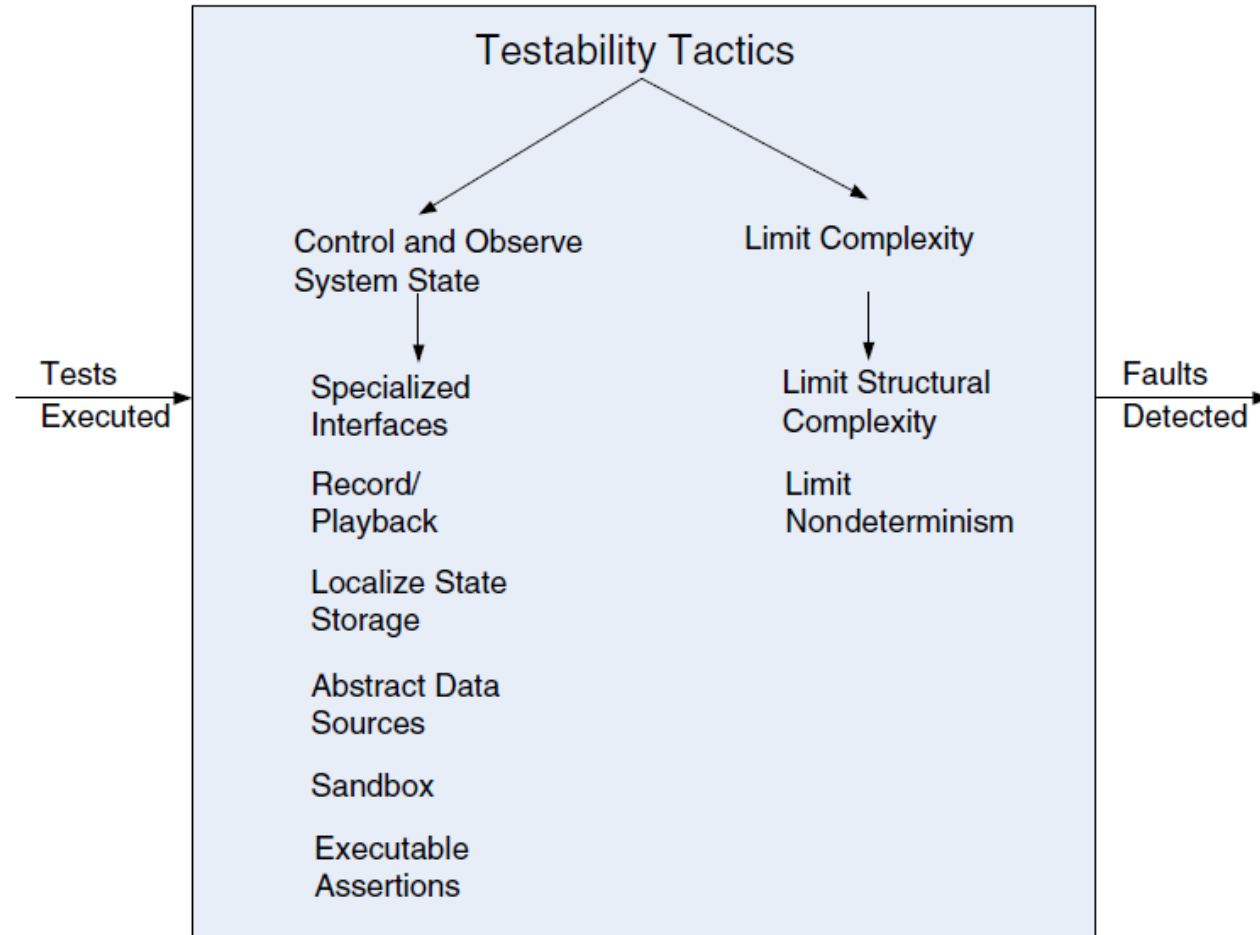
부분	설명	가능한 값
자극원	테스트 케이스는 사람이나 자동화된 테스트 툴에 의해 실행될 수 있다.	다음 중 하나 이상이 가능하다. ▪ 사람 (단위 테스터, 통합 테스터, 시스템 테스터, Acceptance 테스터, 최종 사용자) ▪ 자동화 테스터
자극	테스트 혹은 테스트 집합이 시작된다.	▪ 시스템 기능을 검증 ▪ 품질을 검증 ▪ 품질에 대한 잠재적인 위협을 찾아냄
대상	테스트 중인 시스템의 구성 부분 및 모든 필수 테스트 인프라	테스트 중인 시스템의 부분 ▪ 코드 단위 (아키텍처에서 모듈에 해당하는 부분) ▪ 컴포넌트 ▪ 서비스 ▪ 하위 시스템 ▪ 전체 시스템 ▪ 테스트 인프라

Testability General Scenario (2/2)

부분	설명	가능한 값
환경	어떤 상태에서 자극이 발생하는가?	▪ 클래스나 레이어, 서비스와 같은 코딩 단위를 완료 ▪ 하위 시스템의 통합을 완료 ▪ 전체 시스템의 완전한 구현 ▪ 시스템을 실운영 환경에 배포 ▪ 시스템을 고객에게 배포 ▪ 테스트 스케줄
응답	자극에 대한 반응	▪ 테스트 스위트를 실행하고 결과를 기록 ▪ 결함을 발생시킨 활동을 기록 ▪ 시스템 상태를 제어하고 모니터링
응답 측정	응답을 어떻게 측정하는가?	▪ 결함 혹은 결함의 종류를 찾아내는 데 필요한 노력 ▪ 상태 공간에 대한 커버리지를 몇 퍼센트 달성하라고 주어졌을 때 이를 달성하는 데 필요한 노력 ▪ 어떤 결함이 다음 테스트에서 드러날 확률 ▪ 결함을 감지하는 데 드는 노력 ▪ 테스트 인프라를 준비하는 데 드는 시간의 길이 ▪ 시스템을 특정 상태로 만드는 데 드는 노력 ▪ 위험 노출이 감소하는 정도 : 크기(손실) × 확률(손실)

Testability Tactics

- Tests Executed → Faults Detected : **Control and Observe System State / Limit Complexity**

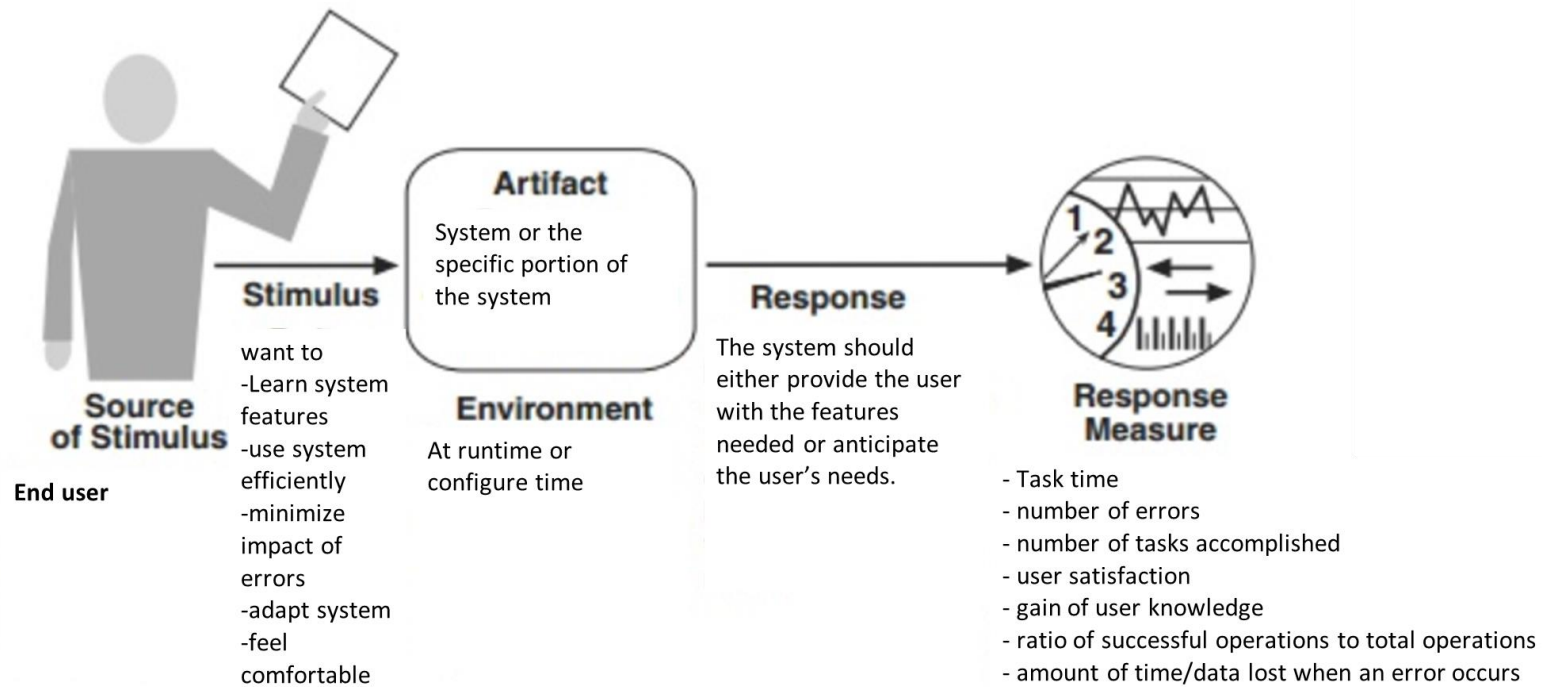


Usability

■ Usability (사용성)

- **how easy it is** for the user to accomplish a desired task 사용자가 원하는 작업을 얼마나 쉽게 수행하는가
- **the kind of user support** the system provides 시스템이 제공하는 사용자 지원의 종류

■ Usability General Scenario



Usability Scenario 예시

- 같은 Usability 라도 무엇을 측정하려는지에 따라 Response Measure 가 달라진다 — 두 가지 시나리오 예

예 1 · 과제 정보 조회 접근 단계

사용자가 과제의 정보를 조회하고자 할 때, 사용자는 인증을 포함하여 총 3단계 이내에 정보 조회에 접근할 수 있다.

- **Source** : 사용자
- **Stimulus** : 과제 정보 조회
- **Environment** : 정상 운영 환경
- **Artifact** : 시스템
- **Response** : 과제 정보 제공
- **Response Measure** : 접근 과정 ≤ 3단계 (인증 포함)

예 2 · Look & Feel 유사도

사용자가 과제 정보를 Confluence 에서 조회하고자 할 때, Confluence 는 GoJIRA 에서 제공되는 UX 와 90% 이상 동일한 Look & Feel 을 제공한다.

- **Source** : 사용자
- **Stimulus** : 과제 정보 조회
- **Environment** : Confluence 시스템 사용
- **Artifact** : Confluence 시스템
- **Response** : GoJIRA 와 동일한 UX 제공
- **Response Measure** : Look & Feel 유사도 ≥ 0.9

※ Look & Feel 유사도 = 동일한 화면 구성 요소 개수 / 전체 화면 구성 요소 개수

Usability 측정 사례 — Look & Feel 유사도 (LPIPS)

- Document Editor & PDF Conversion 평가 : 입력 화면과 출력 문서의 **Look & Feel 유사도** 를 LPIPS 로 측정

SDP (Sensor Design Platform, S.LSI) 시스템 적용/운영 중 (~25.9) — 국내 Sensor 개발팀 및 해외 연구소 (SIRC) 대상 서비스 Open 및 운영 중

1. 입력 화면 / 출력 문서 화면 캡처

※ Word 문서

Instance		RESETGEN_SECU			Scan Reset	Sync/A sync.
Name	Input Port	RESETGEN_SECU Output		clock domain	Reset	
ADG_AH8_SECU_CPU	HRESETn_in	reset_n_sckr_ahb	SCLK (PLL clock)	HRESET n	L_TEST_SCAN _RESET	Async.
ADG_AH8_SECU_JSP	HRESETn_in	reset_n_sckr_ahb	SCLK (PLL clock)	HRESET n	L_TEST_SCAN _RESET	Async.
AH8APR4_BRIDGE_SECU	HRESETn_in	reset_n_sckr_ahb	SCLK (PLL clock)	HRESET n	L_TEST_SCAN _RESET	Async.
AH8_BUSMATRIX_SECU	HRESETn_in	reset_n_sckr_ahb	SCLK (PLL clock)	HRESET n	L_TEST_SCAN _RESET	Async.
BIRA_SYNC_CAPTURE_SECU	reset_n_mclk	reset_n_mclk	MCLK (ext. clock)	HRESET n @MCLK	L_TEST_SCAN _RESET	Async.

※ PDF 문서

Instance		RESETGEN_SECU		Scan Reset	Sync./Asy. Bc.
Name	Input Port	RESETGEN_SECU Output	clock domain	Reset	
ADG_AH8_SECU_CPU	HRESETn_in	reset_n_sckr_ahb	SCLK (PLL clock)	HRESETn	LTEST_SCAN_RESET
ADG_AH8_SECU_JSP	HRESETn_in	reset_n_sckr_ahb	SCLK (PLL clock)	HRESETn	LTEST_SCAN_RESET
AH8APR4_BRIDGE_SECU	HRESETn_in	reset_n_sckr_ahb	SCLK (PLL clock)	HRESETn	LTEST_SCAN_RESET
AH8_BUSMATRIX_SECU	HRESETn_in	reset_n_sckr_ahb	SCLK (PLL clock)	HRESETn	LTEST_SCAN_RESET
BIRA_SYNC_CAPTURE_SECU	reset_n_mclk	reset_n_mclk	MCLK (int. clock)	HRESETn @MCLK	LTEST_SCAN_RESET

2. LPIPS 평가 (python)

```
import torch
import lpips
import cv2
import numpy as np

# LPIPS 모델 로드 (vgg, AlexNet 등 선택 가능)
loss_fn = lpips.LPIPS(net='vgg')

def load_image(path):
    # OpenCV로 이미지 읽고 sRGB 변환, float32로 변환 후 [-1, +1] 범위 정규화
    img = cv2.imread(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = img.astype(np.float32) / 127.5 - 1.0
    img = torch.tensor(img).permute(2, 0, 1).unsqueeze(0) # NCHW 형태
    return img

def compare_image(file1, file2):
    img1 = load_image(file1)
    img2 = load_image(file2)

    # 이미지 크기 맞춤 (필요시 리사이징)
    img2 = torch.nn.functional.interpolate(img2, size=img1.shape[2:], mode='bilinear')

    # LPIPS 거리 계산 (값이 낮을수록 유사)
    d = loss_fn(img1, img2)
    print(f'LPIPS Distance: {d.item():.10f}, image1: {file1}, image2: {file2}')

files = [
    'B0B_TEST_PUNCI_wordedit_pdf1.png', 'B0B_TEST_PUNCI_wordedit_word1.png',
]

for file1, file2 in files:
    compare_image(file1, file2)
```

※ LPIPS : 2개 이미지의 유사도 평가에 사용되는 지표. 0~1 범위이며, 0에 가까울수록 더 유사하다.

3. 평가 결과

```
LPIPS Distance: 0.26754647493362427
LPIPS Distance: 0.1585569679737091
LPIPS Distance: 0.1585569679737091
LPIPS Distance: 0.0
LPIPS Distance: 0.2531733512878418
Average LPIPS Similarity: 0.1423
```

LPIPS : 0.14
유사도 → 86%

4. 문서 출력 시간 최소화

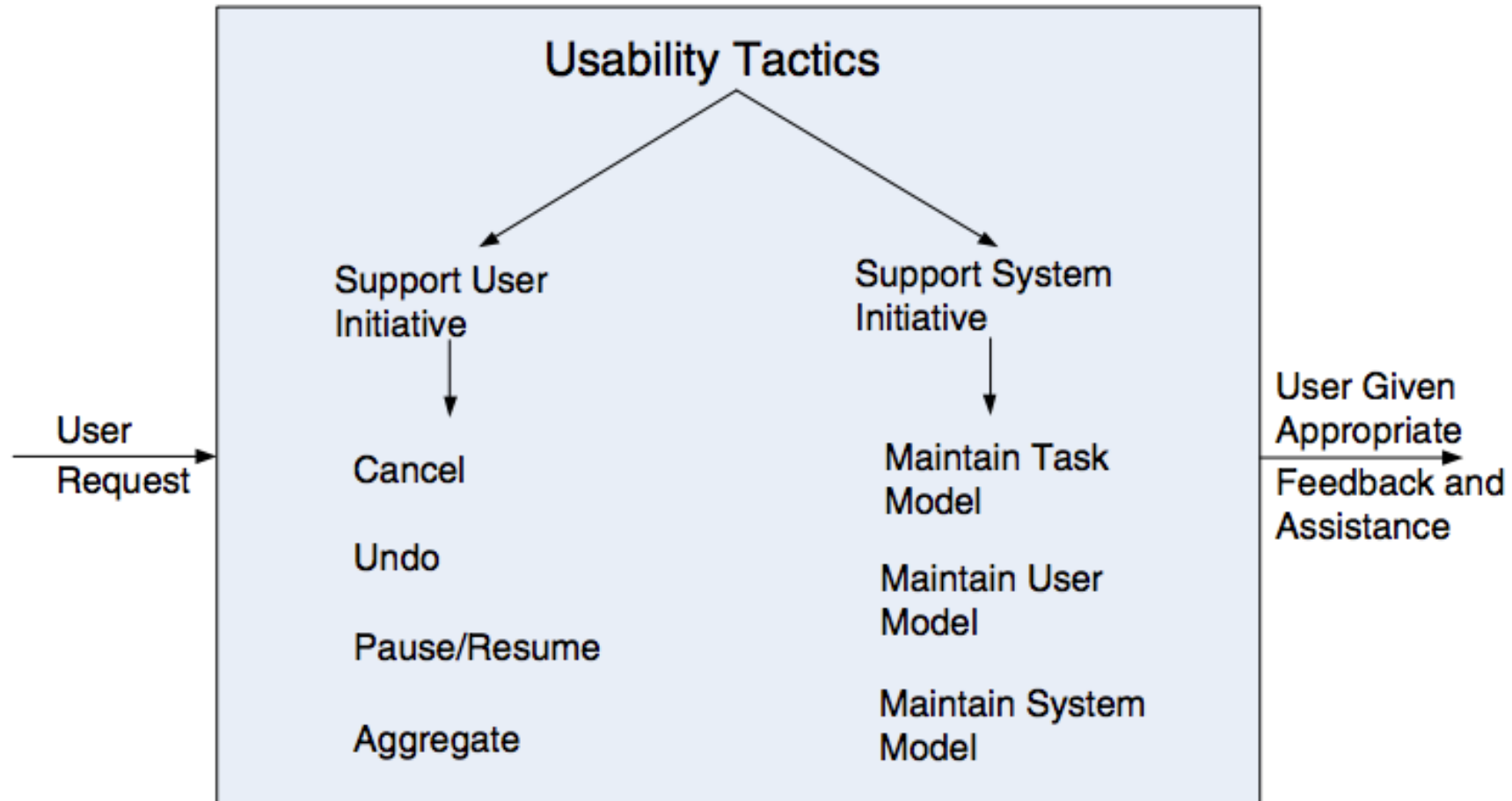
X	Headers	Payload	Preview	Response	Initiator	Timing
Queued at 91.07 ms						
Started at 96.25 ms						
Resource Scheduling						DURATION
Queueing						4.59 ms
Connection Start						DURATION
Stalled						0.77 ms
Proxy negotiation						1 μs
Request/Response						DURATION
Request sent						80 μs
Waiting for server response						8.90 s
Content Download						3.04 ms
Explanation						8.90 s

시스템 동작 검증
PDF 출력 : 8.9초

위: Word 문서 / 아래: PDF 문서

Usability Tactics

- User Request → User Given Appropriate Feedback and Assistance : **Support User Initiative** / **Support System Initiative**



Lunch Time

12:30 ~ 13:30

시간	4일차
9:30	3.10 Testability Tactics (17min) / 3.11 Usability Tactics (12min) 3.12 Tactics and Architectural Patterns (21min)
10:00	
10:30	Ex 6 발표 및 피드백
11:00	Ex 7. Architecture Tactic 실습 2 (Performance~Usability 중심설계)
11:30	
12:00	
12:30	점심 식사
13:00	Ex 7 발표 및 피드백
13:30	
14:00	4-1 Architecture Design (22min) 4-2 Architecture Design (34min)
14:30	
15:00	Ex 8. Architecture 설계 실습
15:30	
16:00	
16:30	
17:00	Ex 8 발표 및 피드백
17:30	
18:00	과정 정리

Day 4 · 오후

온라인 영상 학습

14:00 ~ 15:00

- 4-1 Architecture Design (22분)
- 4-2 Architecture Design (34분)

※ 영상은 1.2 배속으로 학습하세요.

시간	4일차
9:30	3.10 Testability Tactics (17min) / 3.11 Usability Tactics (12min) 3.12 Tactics and Architectural Patterns (21min)
10:00	
10:30	Ex 6 발표 및 피드백
11:00	Ex 7. Architecture Tactic 실습 2 (Performance~Usability 중심설계)
11:30	
12:00	
12:30	점심 식사
13:00	Ex 7 발표 및 피드백
13:30	
14:00	4-1 Architecture Design (22min) 4-2 Architecture Design (34min)
14:30	
15:00	Ex 8. Architecture 설계 실습
15:30	
16:00	
16:30	
17:00	Ex 8 발표 및 피드백
17:30	
18:00	과정 정리

Day 4 · 오후

Ex8 실습 시간

15:00 ~ 17:30

- Architecture Design 실습 — Architectural Driver 도출 · 우선순위 평가 · Design Concern/Option 선정 · 기존 설계에 반영 (조별)
- 실습 템플릿: ex8_Architecture Design exercise-template.xlsx

Ex 8

Architecture Design 실습 소개

주어진 시스템의 Architecture Driver 를 도출하고, 이를 만족시키는 설계 안을 탐색·결정한다

- 실습 개요 · 목표 · 진행 절차 · 시간표 · 환경과 결과물
- 실습 관련 이론: Design Concerns · Trade-Off · Attribute-Driven Design (ADD)

Architecture Design 실습 개요

■ 개요

- 주어진 시스템에 대해 가장 중요한 3개의 Architecture 드라이버를 도출하고, 이를 만족시킬 수 있는 설계 안을 탐색하고 결정한다

■ 목표

- Architecture 드라이버 도출 및 우선순위화
- 품질속성을 만족시킬 수 있는 다양한 Architecture Tactic 및 스타일/패턴들에 대한 이해
- Architecture 뷰 작성 방법

■ 실습 내용

- Ex8. Architecture Design 실습

Architecture Design 실습 절차

Ex8. Architecture Design 실습

- 1) Architectural Driver (AD) 도출 : 설계에 영향을 주는 주요 요구사항
 - Functionality (2개 이상)
 - Quality (3개 이상)
 - Constraint (1개 이상)
- 2) Quality 우선순위 평가에 따른 주요 AD 선정 (3개)
 - Importance > Difficulty (예: H/H > H/L > L/H > L/L)
- 3) 주요 AD 별 최적의 Design Concern 선정 : Design Option 2개 이상
 - 주요 AD 를 만족시킬 수 있는 Design Concern/Option 을 도출하고 분석을 통해 최적의 Design 을 선정
 - 연관된 Functionality/Constraint 가 있을 경우 모두 만족해야 한다
- 4) 기존 설계에 반영 (C&C or Module View)
 - 관련된 부분만 Excel 에 그림으로 붙여넣기

(Initial) Architectural Driver				
ID	Detailed Description	Importance Functionality (Mandatory) Optional (선택)	Difficulty Functionality (N/A 표기) (Importance & Difficulty) 조사가 필요	Priority Quality Attribute에 대해서만 우선 순위 표시
AD-FR1	여러 센서에서 측정된 데이터를 기반으로 공이 날아가는 재해를 계산할 수 있어야 한다.	Mandatory	N/A	N/A
AD-FR2	계정의 스크어를 자동으로 기록하고 저장해야 한다.	Mandatory	N/A	N/A
AD-FR3	계정의 스크어에 대해서 과거 기록을 비교 분석 가능해야 한다.	Optional	N/A	N/A
AD-QA1	시스템 정상 운영 중인 상황에서 서버에 Fatal crash가 발생하더라도 서버 요청시 99.9%로 응답을 전달한다. (Availability)	H	H	1
AD-QA2	시스템이 정상 운영 중인 상황에서 신규 센서 디바이스 연동 요청이 있을시, 연동 후 데이터 교환 성공률이 99% 이상 유지해야 한다. (Interoperability)	M	H	4
AD-QA3	시스템이 정상 운영 중인 상황에서 기획자가 새로운 콜프코스 추가 요청시 운영자는 파일 변경 2개 이하, 전체 코드베이스의 3%(4)만 수정으로 코스를 추가해야 한다. (Modifiability)	L	H	6
AD-QA4	시스템이 정상 운영 중인 상황에서 사용자가 타 지역에서 소말을 한 경우 시스템은 빠른 스텝데이터 분석을 완료하고 사용자에게 보여줘야 한다. 이때 시스템은 1초 내로 공의 속도, 방향, 거리 등을 계산하여 사용자에게 보여준다. (Performance)	H	H	2
AD-QA5	시스템이 정상 운영 중인 상황에서 비인가된 사용자가 Data 변경을 시도할 경우, 비인가된 사용자의 접근을 99% 막는다. (Security)	H	M	3

1)2) Architectural Driver 도출 및 우선순위 (Excel 템플릿)

Design Concern			Design Option		Design Rationale	
AD ID	Design Concern	Design Option	Weakness	Design Option	Design Rationale	Design Rationale
AD-QA1	Availability	Heartbeat	Heartbeat	Heartbeat	Heartbeat	Heartbeat
AD-QA2	Interoperability	API	API	API	API	API
AD-QA3	Modifiability	API	API	API	API	API
AD-QA4	Performance	API	API	API	API	API
AD-QA5	Security	API	API	API	API	API

3) Design Concern / Option 선정



4) View 에 반영

Architecture Design 실습 환경과 결과물

■ 실습 환경 및 도구

- 실습 템플릿 : **ex8_Architecture Design exercise-template.xlsx** (Excel 파일)
- 사용 가능한 임의의 UML 모델링 도구

■ 실습 결과물

- 조별 실습 결과물은 Excel 파일 형식으로 제출
- Google Docs 에 그림 추가가 안 될 경우, Design Decision 까지는 Docs 에서 작업 후 다운로드하여 View 그림 추가 후 제출 (카페 게시 또는 메일)

Architecture Design 실습 시간표

실습 활동	시간 (분)
Architecture Design 실습 설명	10
Ex8. Architecture Design 실습	
조별 실습	120
조별 실습 결과에 대한 발표 및 토의	30

Ex8. Architecture 설계

- 배포한 실습 **Template 파일**을 조별로 작성하세요
- **16:50** 까지 Template 파일 작성 (Café 게시)
- 도움이 필요하거나 질문이 있으시면 메시지로 알려 주세요

실습 절차

- **1) Architectural Driver (AD) 도출**
 - Functionality (2개 이상)
 - Quality (3개 이상)
 - Constraint (1개 이상)
- **2) AD 우선순위 평가**
 - H/H > H/L > L/H > L/L
- **3) 주요 AD 별 최적의 Design Concern 선정**
 - Design Option 2개 이상
- **4) 기존 설계에 반영 (C&C or Module View)**

Design Concerns

■ Architectural Drivers : Architecture 설계를 주도(Drive)하는 Requirements

- Constraints
- Quality Attributes
 - 예 : 사용자1이 편집 모드에서 문서를 수정하면 시스템은 수정 내용을 0.1초 이내에 사용자2에게 반영한다
- Primary Functionality 설계에 영향을 미치는 주요 기능 요구사항 — ASR: Architecturally Significant Requirement
 - 예 : 여러 명의 사용자는 동시에 같은 문서를 조회하고 편집할 수 있다

■ Design Purpose / Objective : 설계에서 달성하고자 하는 목적

- 여러 개의 Driver 가 결합하여 하나의 Design Purpose/Objective 를 구성하는 경우도 있다 예 : 사용자의 문서 수정 내용 실시간 동기화

■ Architectural (Design) Concerns : 설계 목적 달성을 위해 고려해야 하는 사항

- 예 : 서버 이벤트를 클라이언트로 전송하는 방식
- 예 : 이벤트 발생 후 0.1초 이내 동기화를 완료하기 위한 방안

Trade-Off

- 모든 Design Solution 은 어떤 QA 에 대해서는 긍정적, 다른 QA 에 대해서는 부정적 (Risk) 으로 동작할 수 있다
→ **Trade-Off 의 발생** (**Architecture Evaluation 필요성**)
- Trade-Off 의 해소 (Mitigation) 를 위하여 추가의 Design Solution 이 필요하다

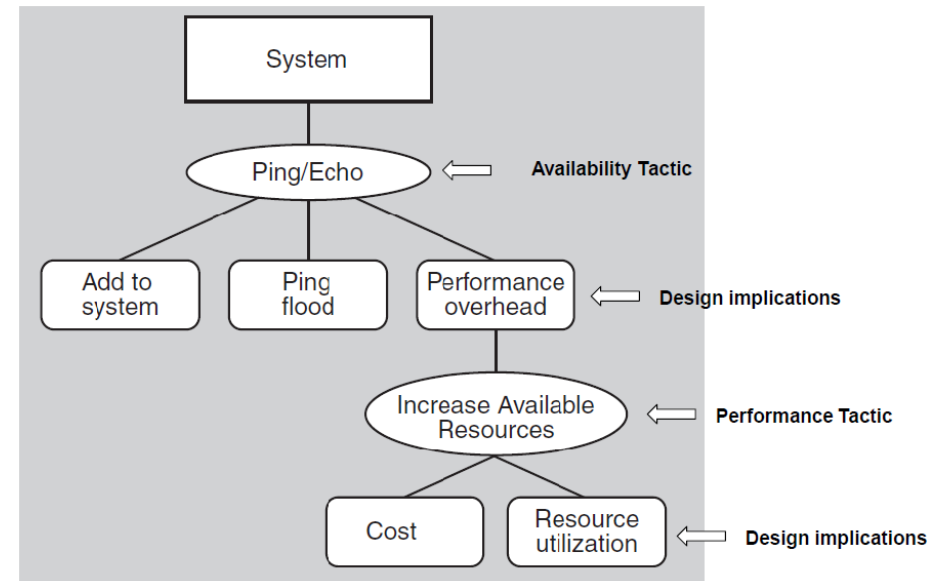
■ 사례 : Broker Pattern 의 취약점

- Availability : **Single point of failure**
- Performance : **Indirection overhead** (latency 증가)

해결 방안 (추가 Tactic 적용)

- ✓ **Maintain multiple copies** (for Performance) : 여러 개의 broker / Load balancing 필요
- ✓ **Ping/Echo** (for Availability) : Broker 문제 감지

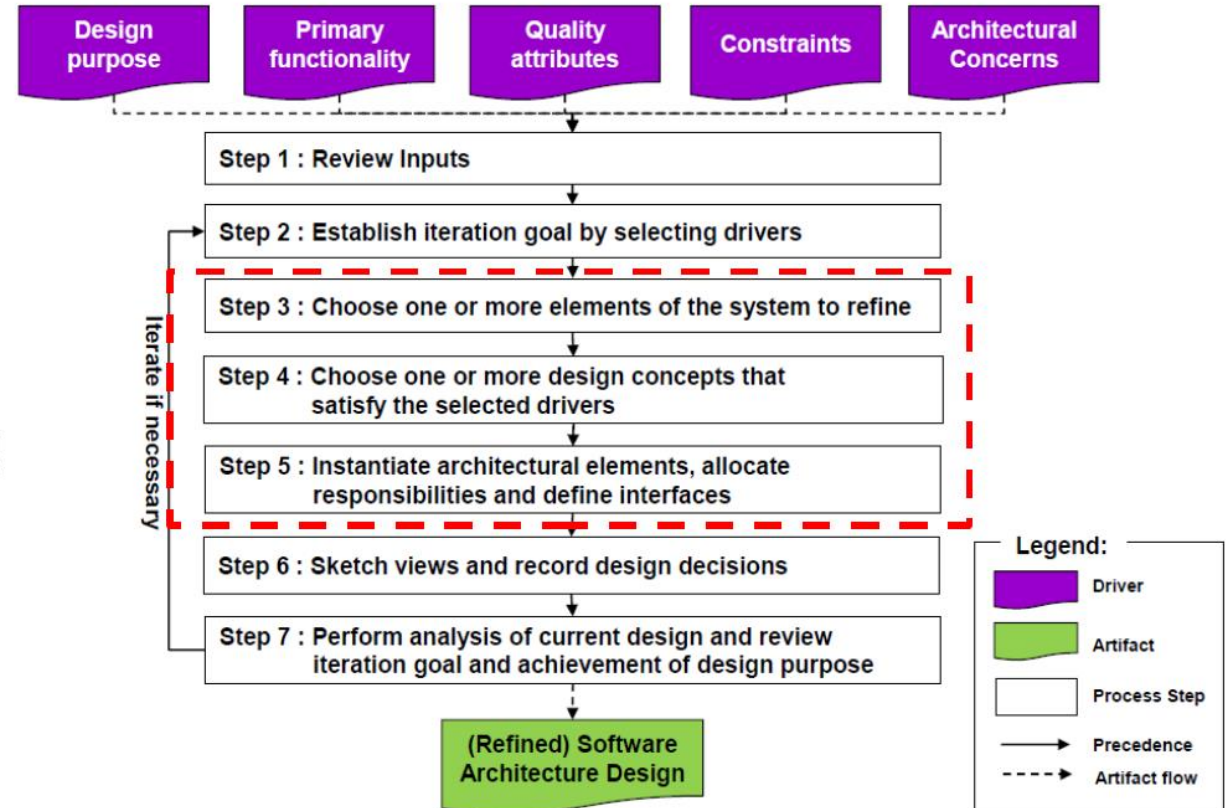
- **Tactic** 은 하나의 QA 에 영향을 미치는 Design Solution
- 모든 Tactic 은 사이드 이펙트를 가지며, 다른 QA 와의 trade-off 를 고려해야 한다



Attribute-Driven Design (ADD)

■ Iterative approach

- **단계 1** : 입력 리뷰
- **단계 2** : 드라이버 (Primary functionality 나 QA) 들 선택을 통한 반복 목표 수립
- **단계 3 / 4 / 5**
 - 시스템에서 정제하고자 하는 하나 이상의 요소 선택
 - 선택한 드라이버들을 만족시키는 하나 이상의 디자인 컨셉 선택
 - 아키텍처 요소들을 실체화하고 책임들을 할당하고 인터페이스들을 정의
- **단계 6** : 뷰들을 스케치하고 설계 결정들을 기록 (rationale)
- **단계 7** : 현재 설계 분석을 수행하고 반복 목표와 설계 목적 달성을 리뷰



Associate Architect 양성과정 · Architecture 설계

Thank You

수고하셨습니다